Déploiement de Last Loop sur un VPS Ubuntu avec Nginx

Guide de production et exploitation

Version 1.0
11 novembre 2026
Cible Ubuntu récent avec Nginx et SQLite

Objet du guide

Ce document décrit une installation complète de Last Loop sur un VPS Ubuntu récent. Le serveur Node.js écoute uniquement sur 127.0.0.1. Nginx reçoit le trafic public, gère HTTPS avec Certbot et transmet les requêtes à l'application. La base SQLite, les secrets et les sauvegardes sont conservés hors du dossier des versions.

Le résultat attendu est un service qui redémarre automatiquement, une URL publique HTTPS, une base persistante, des e-mails d'invitation fonctionnels et une procédure documentée de sauvegarde, mise à jour et retour arrière.

Architecture cible

Flux de production

```
Internet
  |
  v
Nginx  ports 80 et 443  certificat TLS
  |  proxy HTTP avec X Forwarded Proto
  v
Last Loop  127.0.0.1:3000  service systemd
  |
  +-- SQLite  /var/lib/last-loop/backyard.sqlite
  +-- SMTP externe
  +-- Fournisseur de tuiles OpenStreetMap
```

Sommaire opérationnel

1. Préparer le domaine, le VPS et l'accès SSH
2. Mettre Ubuntu à jour et installer les outils système
3. Installer Node.js 22 et Nginx
4. Créer l'utilisateur système et les répertoires persistants
5. Livrer le code et construire la version de production
6. Configurer les variables d'environnement
7. Créer le service systemd
8. Configurer Nginx et obtenir le certificat HTTPS
9. Activer le pare-feu
10. Initialiser Last Loop et tester les fonctions métier
11. Automatiser les sauvegardes et préparer la restauration
12. Mettre à jour, superviser et dépanner le service

Valeurs à personnaliser

Les commandes utilisent les variables ci-dessous. Remplace chaque exemple avant le déploiement. Le guide suppose que la branche de production est main.

Variable	Exemple	Usage
LL_DOMAIN	backyard.example.fr	Nom DNS public de l'application
LL_ADMIN_EMAIL	admin@example.fr	Adresse utilisée par Certbot
LL_GIT_REPO	git@github.com:organisation/last-loop.git	Dépôt contenant le projet
LL_BRANCH	main	Branche déployée
LL_VPS	deploy@203.0.113.10	Connexion SSH utilisée pour rsync

Variables de la session SSH

```
export LL_DOMAIN="backyard.example.fr"
export LL_ADMIN_EMAIL="admin@example.fr"
export LL_GIT_REPO="git@github.com:organisation/last-loop.git"
export LL_BRANCH="main"
```

Préconditions

- Un VPS Ubuntu 24.04 LTS ou version plus récente avec au moins 1 Go de RAM et 2 Go recommandés.
- Un utilisateur SSH disposant de sudo. La connexion directe de root n'est pas nécessaire.
- Un enregistrement DNS A pointant vers l'adresse IPv4 du VPS. Ajouter AAAA uniquement si IPv6 fonctionne réellement.
- Les ports TCP 80 et 443 accessibles depuis Internet. Le port 3000 ne doit jamais être ouvert publiquement.
- Un serveur SMTP capable d'envoyer les invitations et les liens de réinitialisation.
- Un fournisseur de tuiles OpenStreetMap adapté au volume attendu ou l'acceptation explicite de la politique du serveur communautaire.

Préparation du VPS

Connexion et contrôle du système

Depuis le poste administrateur

```
ssh deploy@ADRESSE_IP_DU_VPS
lsb_release -a
uname -m
free -h
df -h /
```

Une architecture amd64 ou arm64 convient. Vérifie que la partition système conserve plusieurs gigaoctets libres pour Node.js, les releases, les journaux et les sauvegardes.

Mise à jour Ubuntu

Commandes à exécuter sur le VPS

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y curl ca-certificates gnupg git rsync sqlite3 nginx ufw unattended-upgrades
sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
```

Si le noyau ou des bibliothèques critiques ont été mis à jour, redémarre puis reconnecte-toi avant de continuer.

Redémarrage si nécessaire

```
sudo reboot
```

Mises à jour de sécurité automatiques

Activation et vérification

```
sudo dpkg-reconfigure --priority=low unattended-upgrades
systemctl status unattended-upgrades --no-pager
```

Installation de Node.js 22

Last Loop exige Node.js 22.13.0 ou supérieur. Le dépôt NodeSource 22.x fournit les paquets nécessaires pour Ubuntu récent.

Installation et contrôle

```
curl -fsSL https://deb.nodesource.com/setup_22.x -o /tmp/nodesource_setup.sh
less /tmp/nodesource_setup.sh
sudo -E bash /tmp/nodesource_setup.sh
sudo apt install -y nodejs
node --version
npm --version
command -v node
```

Le chemin affiché par command -v node sera généralement /usr/bin/node. Utilise ce chemin dans le service systemd si le résultat diffère.

Compte système et stockage persistant

Le processus applicatif utilise un compte sans mot de passe ni shell interactif. Les releases sont séparées de la base et des sauvegardes.

Création des identités et répertoires

```
sudo useradd --system --user-group --home-dir /opt/last-loop --shell /usr/sbin/nologin lastloop
sudo install -d -o lastloop -g lastloop -m 0750 /opt/last-loop
sudo install -d -o lastloop -g lastloop -m 0750 /opt/last-loop/releases
sudo install -d -o lastloop -g lastloop -m 0750 /var/lib/last-loop
sudo install -d -o root -g root -m 0700 /var/backups/last-loop
id lastloop
ls -ld /opt/last-loop /opt/last-loop/releases /var/lib/last-loop /var/backups/last-loop
```

Si useradd indique que le compte existe déjà, vérifie-le avec id lastloop et ne recrée pas le compte.

Livraison du code avec Git

Cette méthode convient lorsque le VPS peut lire le dépôt. Pour un dépôt privé, ajoute une clé de déploiement en lecture seule au compte lastloop ou utilise la méthode rsync décrite ensuite.

Clone et build initial

```
export LL_RELEASE_ID="$(date +%Y%m%d%H%M%S)"
export LL_RELEASE_DIR="/opt/last-loop/releases/${LL_RELEASE_ID}"
sudo -u lastloop git clone --depth 1 --branch "$LL_BRANCH" "$LL_GIT_REPO" "$LL_RELEASE_DIR"
sudo -u lastloop npm --prefix "$LL_RELEASE_DIR" ci
sudo -u lastloop env \
  NEXT_PUBLIC_MAP_TILE_URL='https://tile.openstreetmap.org/{z}/{x}/{y}.png' \
  NEXT_PUBLIC_MAP_ATTRIBUTION='&copy; OpenStreetMap contributors' \
  npm --prefix "$LL_RELEASE_DIR" run build
sudo ln -sfn "$LL_RELEASE_DIR" /opt/last-loop/current
readlink -f /opt/last-loop/current
test -f /opt/last-loop/current/dist/standalone/server.js
```

Livraison du code avec rsync

Utilise cette variante lorsque le dépôt reste uniquement sur le poste de développement. La première commande s'exécute sur le poste local depuis la racine du projet.

Sur le poste de développement

```
export LL_VPS="deploy@203.0.113.10"
export LL_RELEASE_ID="$(date +%Y%m%d%H%M%S)"
rsync -az --exclude=.git --exclude=node_modules --exclude=dist --exclude=data --exclude=apps/tracker/node_modules ./ "$LL_VPS:/tmp/last-loop-$LL_RELEASE_ID/"
```

Sur le VPS après le transfert

```
export LL_RELEASE_ID="2 026 111 115 3000"
export LL_RELEASE_DIR="/opt/last-loop/releases/${LL_RELEASE_ID}"
sudo install -d -o lastloop -g lastloop -m 0750 "$LL_RELEASE_DIR"
sudo rsync -a "/tmp/last-loop-$LL_RELEASE_ID/" "$LL_RELEASE_DIR/"
sudo chown -R lastloop:lastloop "$LL_RELEASE_DIR"
sudo -u lastloop npm --prefix "$LL_RELEASE_DIR" ci
sudo -u lastloop env \
  NEXT_PUBLIC_MAP_TILE_URL='https://tile.openstreetmap.org/{z}/{x}/{y}.png' \
  NEXT_PUBLIC_MAP_ATTRIBUTION='&copy; OpenStreetMap contributors' \
  npm --prefix "$LL_RELEASE_DIR" run build
sudo ln -sfn "$LL_RELEASE_DIR" /opt/last-loop/current
```

Le numéro LL_RELEASE_ID utilisé sur le VPS doit être exactement celui créé sur le poste local.

Configuration de production

Le fichier /etc/last-loop.env contient les secrets et les réglages de fonctionnement. Il n'est ni placé dans Git ni copié dans une release.

Création du fichier

```
sudo install -o root -g root -m 0600 /dev/null /etc/last-loop.env
sudo nano /etc/last-loop.env
```

Contenu de /etc/last-loop.env

```
NODE_ENV=production
HOST=127.0.0.1
PORT=3000
DATABASE_PATH=/var/lib/last-loop/backyard.sqlite
SEED_DEMO_DATA=false
APP_URL=https://backyard.example.fr

SMTP_HOST=smtp.example.fr
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=backyard@example.fr
SMTP_PASSWORD=REMPLACER_PAR_UN_SECRET_LONG
MAIL_FROM=Last Loop <no-reply@example.fr>
MAIL_PREVIEW=false

NEXT_PUBLIC_MAP_TILE_URL=https://tile.openstreetmap.org/{z}/{x}/{y}.png
NEXT_PUBLIC_MAP_ATTRIBUTION=&copy; OpenStreetMap contributors

GPS_RETENTION_DAYS=14
GPS_OFF_ROUTE_THRESHOLD_M=200
```

Remplace backyard.example.fr par le domaine réel. APP_URL ne doit pas se terminer par un slash. SMTP_SECURE vaut true avec TLS direct, généralement sur le port 465, et false avec STARTTLS, généralement sur le port 587.

Protection des secrets

```
sudo chown root:root /etc/last-loop.env
sudo chmod 600 /etc/last-loop.env
sudo stat -c '%U %G %a %n' /etc/last-loop.env
```

Variables essentielles

Variable	Valeur recommandée	Contrôle
NODE_ENV	production	Désactive les données de démonstration par défaut
HOST	127.0.0.1	Empêche un accès direct au processus Node
DATABASE_PATH	/var/lib/last-loop/backyard.sqlite	Chemin persistant hors des releases
SEED_DEMO_DATA	false	Ne charge aucun événement fictif
APP_URL	https://domaine	Produit des liens e-mail publics valides
MAIL_PREVIEW	false	Obligatoire en production
GPS_RETENTION_DAYS	14	Plage acceptée de 1 à 90 jours
GPS_OFF_ROUTE_THRESHOLD_M	200	Plage acceptée de 50 à 2000 mètres

Service systemd

systemd démarre Last Loop au boot, redémarre le processus après une erreur et limite ses droits d'écriture au répertoire de la base.

Création du service

```
sudo nano /etc/systemd/system/last-loop.service
```

Contenu de /etc/systemd/system/last-loop.service

```
[Unit]
Description=Last Loop Backyard tracking
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=lastloop
Group=lastloop
WorkingDirectory=/opt/last-loop/current
EnvironmentFile=/etc/last-loop.env
ExecStart=/usr/bin/node /opt/last-loop/current/dist/standalone/server.js
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
KillSignal=SIGTERM
UMask=0077
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
ReadWritePaths=/var/lib/last-loop
LimitNOFILE=65 535

[Install]
WantedBy=multi-user.target
```

Si command -v node n'a pas retourné /usr/bin/node, remplace ExecStart par le chemin réel.

Activation et test local

```
sudo systemctl daemon-reload
sudo systemctl enable --now last-loop
sudo systemctl status last-loop --no-pager
sudo journalctl -u last-loop -n 100 --no-pager
curl -fsS http://127.0.0.1:3000/api/health
```

La dernière commande doit retourner un objet JSON contenant status ok et database ok. Un code HTTP 503 indique que la base n'a pas pu être ouverte.

Configuration Nginx

Commence avec un virtual host HTTP. Certbot utilisera ce bloc pour vérifier le domaine puis ajoutera la configuration TLS et la redirection HTTPS.

Création du virtual host

```
sudo nano /etc/nginx/sites-available/last-loop
```

Contenu de /etc/nginx/sites-available/last-loop

```
server {
    listen 80;
    listen [::]:80;
    server_name backyard.example.fr;

    server_tokens off;
    client_max_body_size 3m;

    add_header X-Content-Type-Options nosniff always;
    add_header Referrer-Policy strict-origin-when-cross-origin always;
    add_header Permissions-Policy "geolocation=(self)" always;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Port $server_port;
        proxy_connect_timeout 5s;
        proxy_send_timeout 90s;
        proxy_read_timeout 90s;
    }
}
```

Activation du site HTTP

```
sudo ln -s /etc/nginx/sites-available/last-loop /etc/nginx/sites-enabled/last-loop
sudo nginx -t
sudo systemctl reload nginx
curl -I http://backyard.example.fr/api/health
```

Ne supprime le site Nginx par défaut que si aucun autre site ne l'utilise. Cette suppression est facultative.

Suppression facultative du site par défaut

```
sudo unlink /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

Certificat HTTPS avec Certbot

Le DNS doit déjà pointer vers le VPS et le port 80 doit être joignable depuis Internet. Certbot recommande son paquet snap pour disposer d'une version maintenue.

Installation et émission du certificat

```
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/local/bin/certbot 2>/dev/null || true
sudo certbot --nginx -d backyard.example.fr -m admin@example.fr --agree-tos --no-eff-email --redirect
```

Contrôles TLS et renouvellement

```
sudo nginx -t
sudo systemctl reload nginx
curl -fsS https://backyard.example.fr/api/health
sudo certbot renew --dry-run
systemctl list-timers --all | grep certbot
```

Après validation complète de HTTPS, ajoute HSTS dans le bloc server qui écoute sur 443, puis teste Nginx. N'active pas HSTS avant d'être certain que le domaine restera disponible en HTTPS.

Directive HSTS à ajouter dans le bloc HTTPS

```
add_header Strict-Transport-Security "max-age=31 536 000" always;
```

Pare feu UFW

Autorise SSH avant d'activer UFW pour éviter de perdre l'accès. Si SSH utilise un port différent, autorise ce port à la place du profil OpenSSH.

Règles recommandées

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status verbose
```

N'ajoute aucune règle pour le port 3000. Le processus Node écoute sur 127.0.0.1 et reste accessible uniquement depuis le VPS.

Premier démarrage applicatif

1. Ouvre https://backyard.example.fr/api/health et vérifie status ok.
2. Ouvre https://backyard.example.fr/setup et crée immédiatement le premier compte directeur.
3. Connecte-toi à l'espace directeur et crée les comptes administrateurs nécessaires.
4. Envoie une invitation à une adresse de test pour vérifier SMTP, APP_URL et la réinitialisation du mot de passe.
5. Crée un événement avec un fichier GPX puis vérifie la distance, le dénivelé et la carte OpenStreetMap.
6. Ouvre la page publique, le dashboard directeur et l'écran vidéo projecteur.
7. Depuis un téléphone en 4G ou 5G, ouvre /track, associe un code GPS et contrôle la réception d'une position.

La route /setup accepte uniquement la création du premier utilisateur. Une fois le directeur créé, elle refuse les nouvelles initialisations.

Contrôle SMTP

En production, l'absence de configuration SMTP provoque une erreur lors de l'envoi d'une invitation ou d'un lien de réinitialisation. MAIL_PREVIEW doit rester à false.

- Port 587 avec SMTP_SECURE=false pour STARTTLS dans la majorité des configurations.
- Port 465 avec SMTP_SECURE=true pour TLS direct.
- APP_URL doit correspondre exactement au domaine HTTPS public.
- MAIL_FROM doit être autorisé par le fournisseur SMTP. Configure SPF, DKIM et DMARC sur le domaine d'envoi.

Après toute modification de /etc/last-loop.env

```
sudo systemctl restart last-loop
sudo journalctl -u last-loop -n 100 --no-pager
```

Cartographie

Le serveur communautaire tile.openstreetmap.org est un service sans garantie de disponibilité. Il peut convenir à un test ou à une audience réduite si sa politique est respectée. Pour une course très consultée, configure un fournisseur de tuiles avec des limites et un support adaptés.

Les variables NEXT_PUBLIC_MAP_TILE_URL et NEXT_PUBLIC_MAP_ATTRIBUTION sont intégrées au code client pendant npm run build. Toute modification impose donc une nouvelle compilation et un redémarrage de l'application.

Sauvegarde automatique de SQLite

SQLite utilise le mode WAL. La commande interne .backup crée une sauvegarde cohérente pendant que l'application fonctionne. Le script ci-dessous compresse le fichier et conserve trente jours de sauvegardes locales.

Création du script

```
sudo nano /usr/local/sbin/last-loop-backup
```

Contenu de /usr/local/sbin/last-loop-backup

```
#!/usr/bin/env bash
set -euo pipefail

LL_DB="/var/lib/last-loop/backyard.sqlite"
LL_BACKUP_DIR="/var/backups/last-loop"
LL_TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
LL_TARGET="${LL_BACKUP_DIR}/backyard-${LL_TIMESTAMP}.sqlite"

install -d -o root -g root -m 0700 "$LL_BACKUP_DIR"
sqlite3 "$LL_DB" ".timeout 5000" ".backup '$LL_TARGET'"
gzip -f "$LL_TARGET"
find "$LL_BACKUP_DIR" -type f -name 'backyard-*.sqlite.gz' -mtime +30 -delete
```

Droits et premier test

```
sudo chown root:root /usr/local/sbin/last-loop-backup
sudo chmod 750 /usr/local/sbin/last-loop-backup
sudo /usr/local/sbin/last-loop-backup
sudo ls -lh /var/backups/last-loop
```

Planification systemd

Service de sauvegarde

```
sudo nano /etc/systemd/system/last-loop-backup.service
```

Contenu du service

```
[Unit]
Description=Sauvegarde SQLite Last Loop

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/last-loop-backup
Nice=10
IOSchedulingClass=best-effort
IOSchedulingPriority=7
```

Minuterie quotidienne

```
sudo nano /etc/systemd/system/last-loop-backup.timer
```

Contenu de la minuterie

```
[Unit]
Description=Sauvegarde quotidienne Last Loop

[Timer]
OnCalendar=*-*-* 03:15:00
Persistent=true
RandomizedDelaySec=15m

[Install]
WantedBy=timers.target
```

Activation et vérification

```
sudo systemctl daemon-reload
sudo systemctl enable --now last-loop-backup.timer
sudo systemctl start last-loop-backup.service
sudo systemctl status last-loop-backup.service --no-pager
systemctl list-timers --all | grep last-loop
```

Une sauvegarde conservée uniquement sur le VPS ne protège pas contre la perte du serveur. Copie régulièrement les archives vers un stockage externe chiffré et teste une restauration complète.

Restauration de la base

Choisis une sauvegarde précise, annonce l'interruption puis arrête l'application avant de remplacer la base. Conserve toujours le fichier actuel pour pouvoir revenir en arrière.

Procédure de restauration

```
sudo systemctl stop last-loop
export LL_RESTORE_ID="$(date -u +%Y%m%dT%H%M%SZ)"
sudo install -d -o root -g root -m 0700 "/var/backups/last-loop/pre-restore-$LL_RESTORE_ID"
sudo cp -a /var/lib/last-loop/backyard.sqlite* "/var/backups/last-loop/pre-restore-$LL_RESTORE_ID/"
sudo mv /var/lib/last-loop/backyard.sqlite-wal "/var/backups/last-loop/pre-restore-$LL_RESTORE_ID/backyard.sqlite-wal.original" 2>/dev/null ||
true
sudo mv /var/lib/last-loop/backyard.sqlite-shm "/var/backups/last-loop/pre-restore-$LL_RESTORE_ID/backyard.sqlite-shm.original" 2>/dev/null ||
true
sudo gzip -dc /var/backups/last-loop/backyard-20 261 111T031 500Z.sqlite.gz | sudo tee /var/lib/last-loop/backyard.sqlite >/dev/null
sudo chown lastloop:lastloop /var/lib/last-loop/backyard.sqlite
sudo chmod 600 /var/lib/last-loop/backyard.sqlite
sudo systemctl start last-loop
curl -fsS http://127.0.0.1:3000/api/health
sudo journalctl -u last-loop -n 100 --no-pager
```

Remplace le nom de l'archive par la sauvegarde choisie. La base précédente est copiée dans le dossier pre-restore. Les éventuels fichiers WAL et SHM sont également préservés puis retirés du répertoire actif avant l'écriture de la sauvegarde restaurée.

Mise à jour de Last Loop

Chaque mise à jour crée une nouvelle release. La base et le fichier d'environnement restent en place. La bascule du lien current permet un retour arrière rapide.

Déploiement d'une nouvelle release

```
sudo /usr/local/sbin/last-loop-backup
export LL_RELEASE_ID="$(date +%Y%m%d%H%M%S)"
export LL_RELEASE_DIR="/opt/last-loop/releases/${LL_RELEASE_ID}"
sudo -u lastloop git clone --depth 1 --branch "$LL_BRANCH" "$LL_GIT_REPO" "$LL_RELEASE_DIR"
sudo -u lastloop npm --prefix "$LL_RELEASE_DIR" ci
sudo -u lastloop env \
  NEXT_PUBLIC_MAP_TILE_URL='https://tile.openstreetmap.org/{z}/{x}/{y}.png' \
  NEXT_PUBLIC_MAP_ATTRIBUTION='&copy; OpenStreetMap contributors' \
  npm --prefix "$LL_RELEASE_DIR" run test
sudo ln -sfn "$LL_RELEASE_DIR" /opt/last-loop/current
sudo systemctl restart last-loop
curl -fsS http://127.0.0.1:3000/api/health
curl -fsS "https://$LL_DOMAIN/api/health"
sudo journalctl -u last-loop -n 100 --no-pager
```

npm run test exécute le lint puis le build. La coupure correspond au temps de redémarrage du service après la bascule du lien.

Retour arrière

Retour au code précédent

```
ls –1dt /opt/last-loop/releases/*
sudo ln -sfn /opt/last-loop/releases/RELEASE_PRECEDENTE /opt/last-loop/current
sudo systemctl restart last-loop
curl -fsS http://127.0.0.1:3000/api/health
sudo journalctl -u last-loop -n 100 --no-pager
```

Un retour au code précédent ne restaure pas automatiquement la base. Si la nouvelle version a modifié le schéma de manière incompatible, restaure aussi la sauvegarde réalisée avant la mise à jour.

Supervision et journaux

Contrôle	Commande ou URL	Résultat attendu
Application locale	curl http://127.0.0.1:3000/api/health	HTTP 200 et status ok
Chaîne publique	curl https://domaine/api/health	HTTP 200 et certificat valide
Service Node	systemctl status last-loop	active running
Nginx	nginx -t	syntax is ok et test is successful
Certificats	certbot renew --dry-run	simulation réussie
Sauvegardes	systemctl status last-loop-backup.service	dernier lancement réussi
Disque	df -h / /var/lib/last-loop	espace libre suffisant

Journaux utiles

```
sudo journalctl -u last-loop -f
sudo journalctl -u nginx -n 100 --no-pager
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

Configure une sonde externe toutes les 30 à 60 secondes sur https://domaine/api/health. Déclenche une alerte pour un statut différent de 200, un certificat proche de l’expiration, un disque presque plein ou l’absence de sauvegarde récente.

Sécurité et protection des données

- Expose uniquement SSH, HTTP et HTTPS. Ne publie jamais le port 3000 ni le fichier SQLite.
- Réserve /etc/last-loop.env à root et ne copie jamais son contenu dans un ticket, un dépôt ou une sauvegarde non chiffrée.
- Utilise des mots de passe uniques et active l'authentification SSH par clé. Désactive la connexion SSH par mot de passe lorsque tous les administrateurs ont validé leur clé.
- Conserve MAIL_PREVIEW=false et SEED_DEMO_DATA=false en production.
- Définis une durée de conservation GPS cohérente avec l'information donnée aux coureurs.
- Informe les participants que les positions récentes peuvent apparaître publiquement pendant la course.
- Teste les sauvegardes hors du VPS et protège leur accès comme celui de la base de production.

Dépannage

Symptôme	Diagnostic	Action
502 Bad Gateway	systemctl status last-loop puis journalctl	Redémarrer le service et corriger l'erreur Node
Connexion qui boucle	Vérifier HTTPS et X-Forwarded-Proto	Conserver le header Nginx et vider l'ancien cookie
Erreur SQLite	Contrôler droits et espace disque	chown lastloop et vérifier /var/lib/last-loop
Invitation non envoyée	Lire les journaux SMTP	Corriger SMTP, MAIL_FROM et APP_URL
Carte absente	Inspecter l'URL des tuiles et le navigateur	Recompiler après modification NEXT_PUBLIC_MAP
GPX refusé	Vérifier taille et structure	Fichier inférieur à 2 Mo avec trkpt ou rtept
GPS web refusé	Vérifier le cadenas HTTPS et la permission	Corriger TLS puis autoriser la localisation

Checklist de mise en production

- Le DNS A pointe vers le VPS et AAAA n'existe que si IPv6 est opérationnel.
- Node.js respecte la version minimale 22.13.0.
- Last Loop écoute sur 127.0.0.1:3000 et le port 3000 est fermé publiquement.
- Le service last-loop est enabled et active running.
- Nginx transmet Host, X-Forwarded-For et X-Forwarded-Proto.
- Le certificat HTTPS est valide et certbot renew --dry-run réussit.
- UFW autorise SSH et Nginx Full uniquement.
- La base réside dans /var/lib/last-loop et appartient à lastloop.
- SEED_DEMO_DATA=false et MAIL_PREVIEW=false.
- Le premier compte directeur a été créé sur /setup.
- Une invitation e-mail et une réinitialisation de mot de passe ont été testées.
- Un GPX réel s'affiche sur la carte publique.
- Un téléphone en réseau mobile envoie une position GPS.
- La sauvegarde quotidienne fonctionne et une copie externe existe.
- La procédure de restauration a été testée hors production.
- Une sonde externe surveille /api/health.

Références officielles

- Distribution Node.js 22 pour Debian et Ubuntu
- Installation de Nginx sur Ubuntu Server
- Pare feu UFW sur Ubuntu Server
- Instructions Certbot pour Nginx
- Module reverse proxy Nginx
- Politique des tuiles OpenStreetMap

Les commandes de ce guide correspondent à l'application Last Loop présente dans ce dépôt et aux procédures officielles consultées le 11 novembre 2026.